E-COMMERCE SALE ANALYSIS

(1)Import the Dataset

```
# Step 1: Import libraries
import pandas as pd
import numpy as np

# Step 2: Load CSV files
orders = pd.read_csv("olist_orders_dataset.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
sellers = pd.read_csv("olist_sellers_dataset.csv")
payments = pd.read_csv("olist_order_payments_dataset.csv")
reviews = pd.read_csv("olist_order_reviews_dataset.csv")

# Step 3: Display info
print("Orders:", orders.shape)
print("Order Items:", order_items.shape)
print("Customers:", customers.shape)
print("Payments:", payments.shape)
print("Reviews:", reviews.shape)

# Step 4: Quick preview of one table
orders.head()
```

```
Orders: (99441, 8)
Order Items: (112650, 7)
Customers: (99441, 5)
Payments: (103886, 5)
Reviews: (99224, 7)
```

Microsoft Edge

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	o
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:06:29	
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59	2017-11-22 13:39:59	2017-12-02 00:28:42	
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29	2018-02-14 19:46:34	2018-02-16 18:17:02	

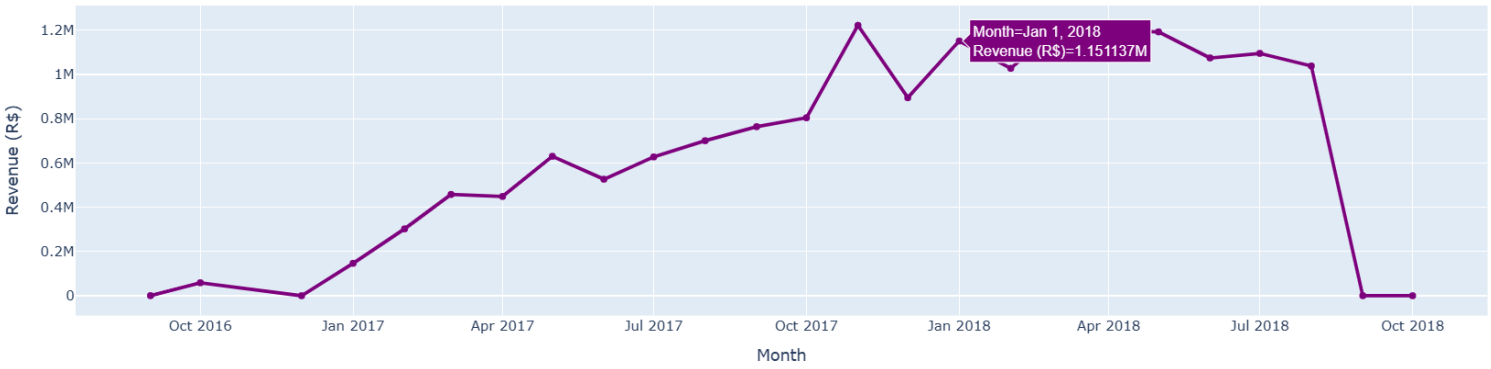
(2)Data Cleaning and Preprocessing

```
# Step 1 : Convert date columns to datetime
date_cols = ['order_purchase_timestamp', 'order_approved_at',
             'order_delivered_carrier_date', 'order_delivered_customer_date',
             'order_estimated_delivery_date']
for col in date_cols:
    orders[col] = pd.to_datetime(orders[col])

# Step 2: Handle missing values (check summary)
orders.isnull().sum()

# Optionally drop rows where essential fields are missing
orders = orders.dropna(subset=['order_purchase_timestamp', 'customer_id'])
```

```python
# Step 3: Merge tables
# Merge orders + customers
order_customer = pd.merge(orders, customers, on='customer_id', how='left')

# Merge with payments
order_customer_pay = pd.merge(order_customer, payments, on='order_id', how='left')

# Merge with order_items
order_full = pd.merge(order_customer_pay, order_items, on='order_id', how='left')

# Merge with products and sellers
order_full = pd.merge(order_full, products, on='product_id', how='left')
order_full = pd.merge(order_full, sellers, on='seller_id', how='left')

print("Final merged dataset shape:", order_full.shape)

# Step 4 : Derive useful columns
order_full['total_price'] = order_full['price'] + order_full['freight_value']
order_full['delivery_days'] = (
    order_full['order_delivered_customer_date'] - order_full['order_purchase_timestamp']
).dt.days

# Step 5 : Handle negative or missing delivery days
order_full['delivery_days'] = order_full['delivery_days'].apply(lambda x: x if x is None or x > 0 else np.nan)

# Step 6 :Check dataset after cleaning
order_full.head()
```

```
Final merged dataset shape: (118434, 33)
```

📈 Monthly Sales Trend (Interactive)



	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	ord
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	
1	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	
2	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	
3	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	
4	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:06:29	

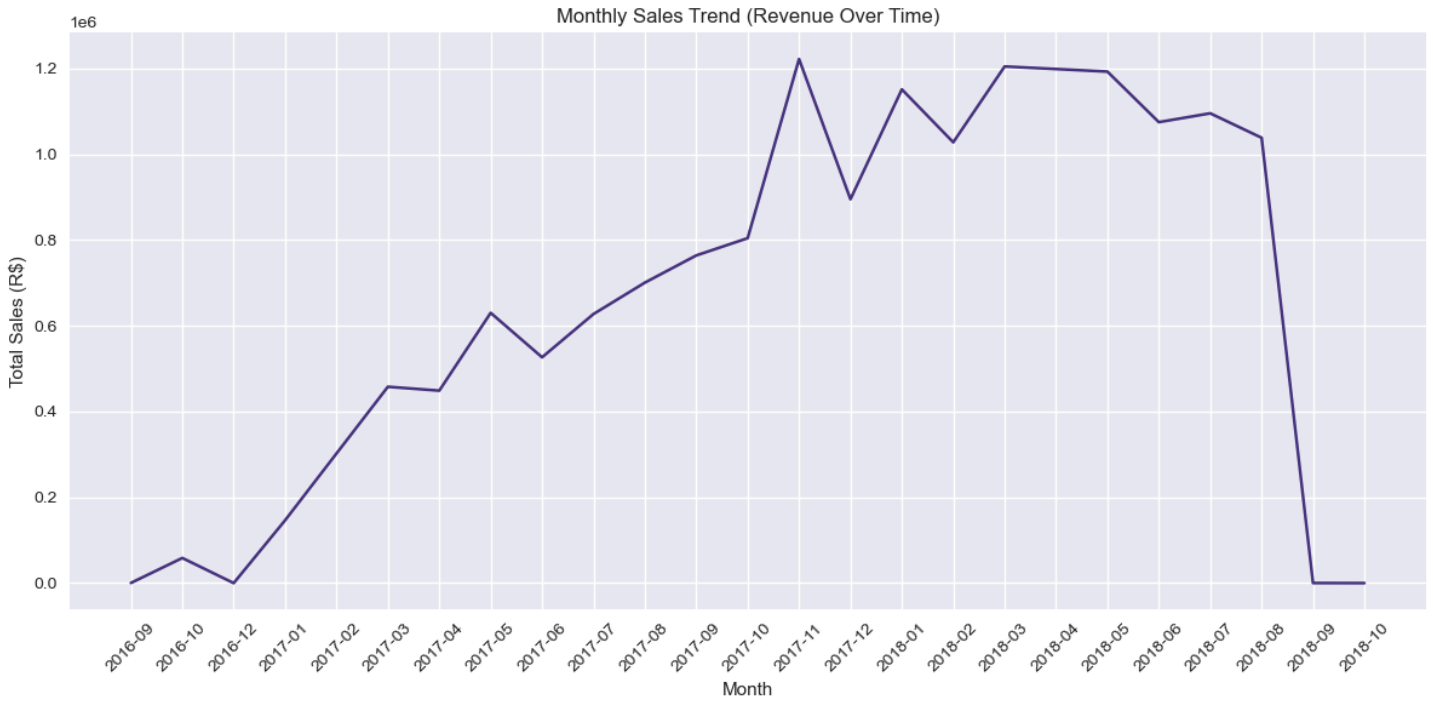
5 rows × 35 columns

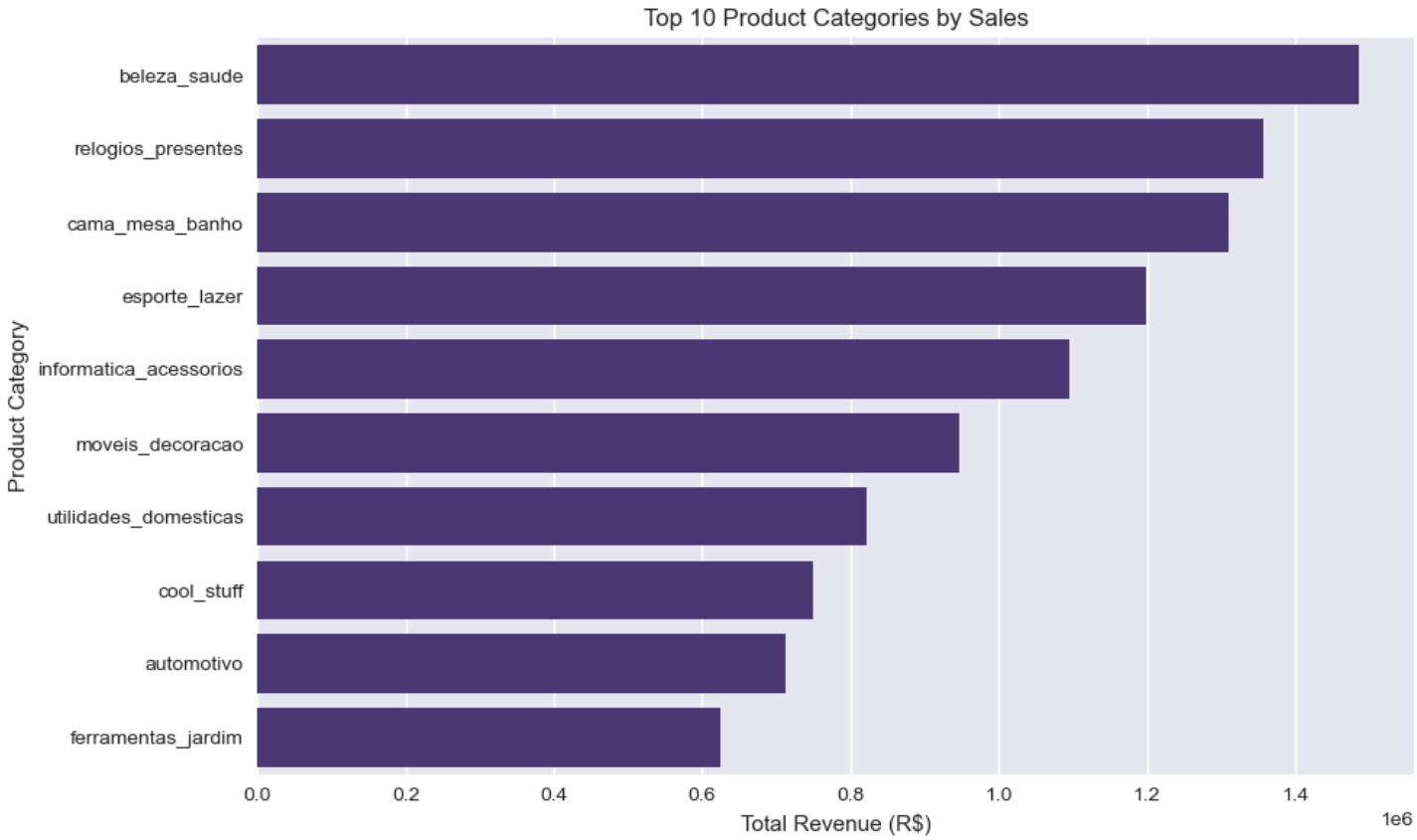
(3)EDA and Visualizations

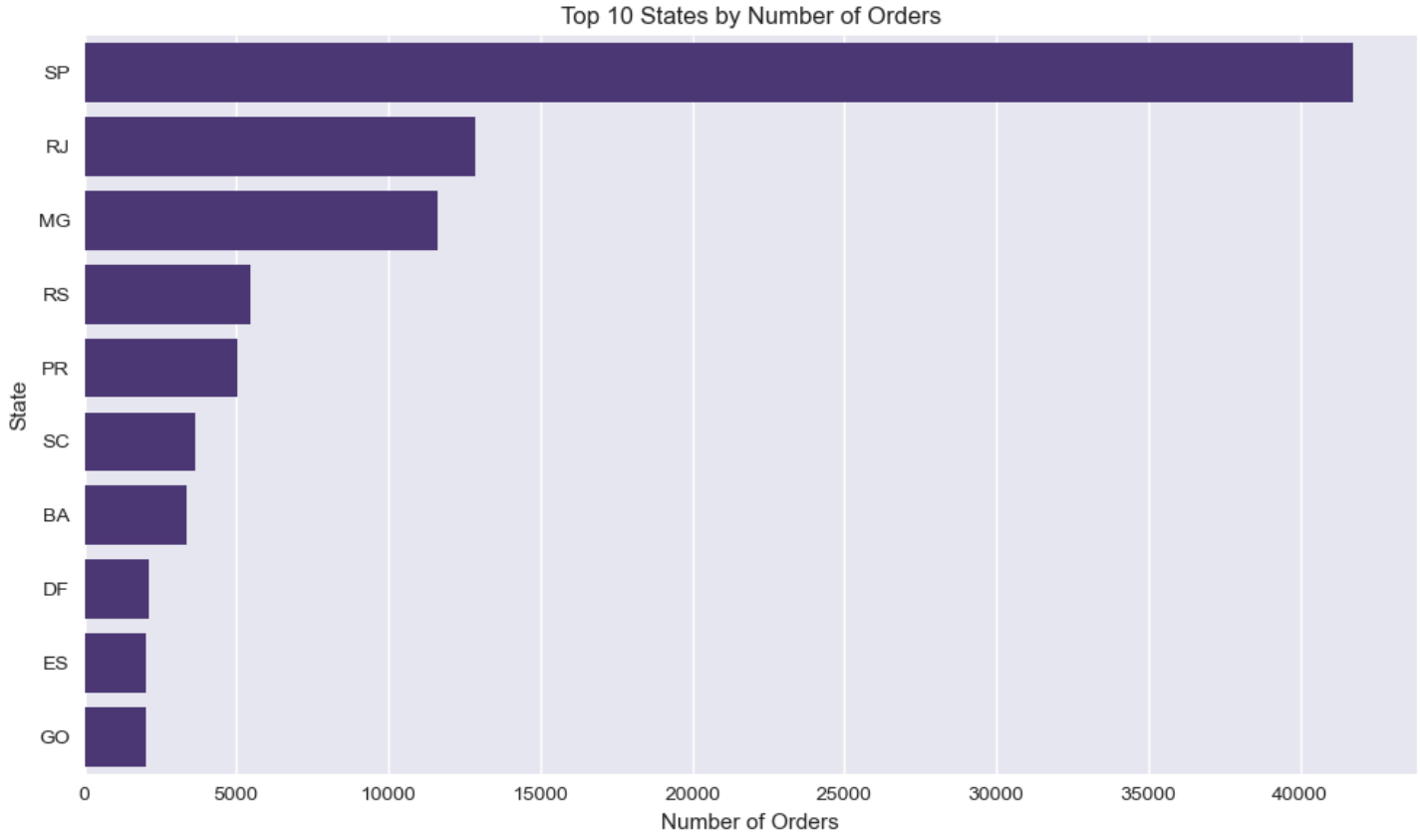
```
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Basic setup
plt.style.use("seaborn-v0_8")
sns.set_palette("viridis")

# Step 1 : Overall sales trend over time
sales_per_month = (
    order_full.groupby(order_full['order_purchase_timestamp'].dt.to_period('M'))['total_price']
```

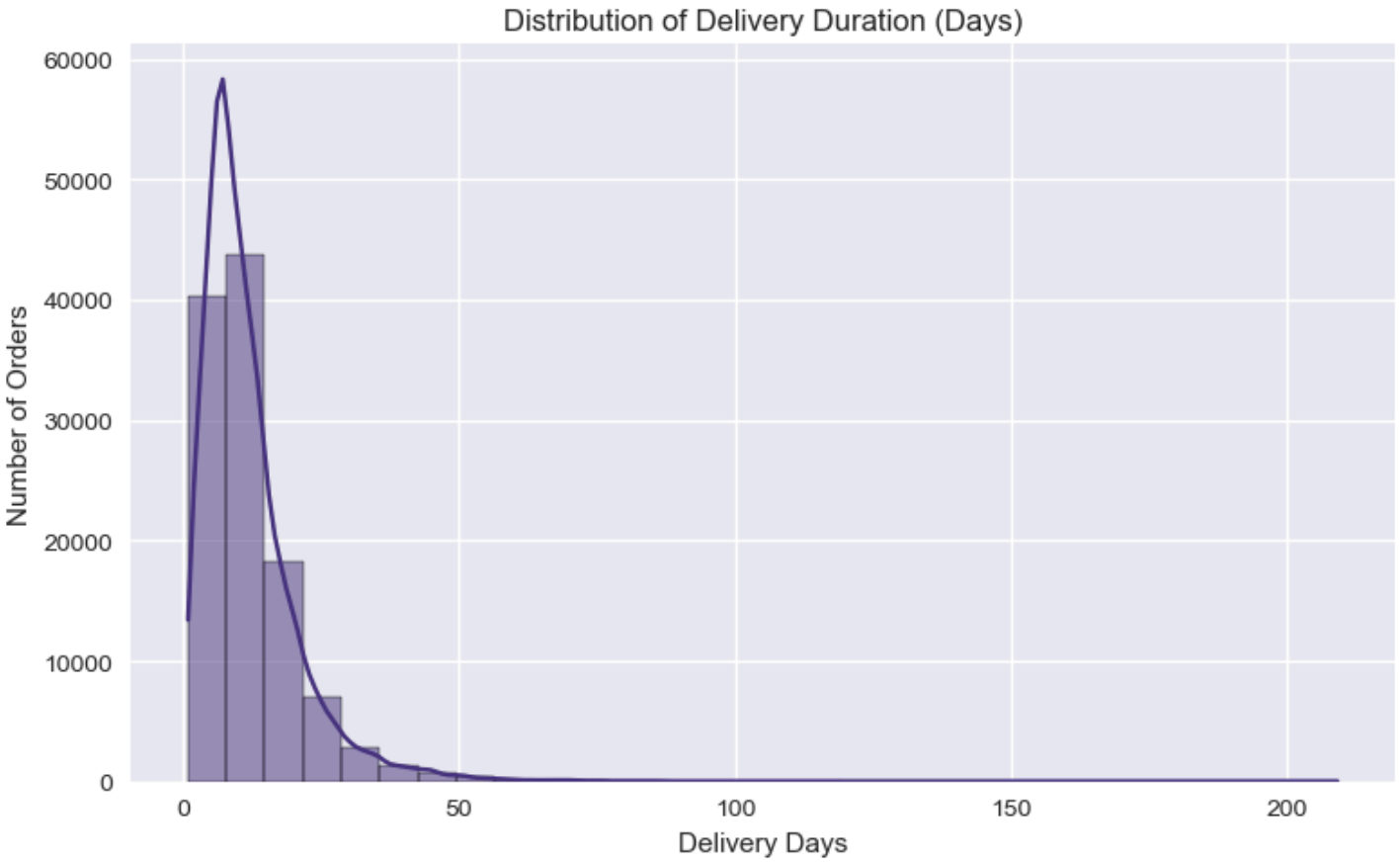
```
sales_per_month = (
    order_full.groupby(order_full['order_purchase_timestamp'].dt.to_period('M'))['total_price']
    .sum()
    .reset_index()
)
sales_per_month['order_purchase_timestamp'] = sales_per_month['order_purchase_timestamp'].astype(str)

plt.figure(figsize=(12, 6))
sns.lineplot(data=sales_per_month, x='order_purchase_timestamp', y='total_price')
plt.title("Monthly Sales Trend (Revenue Over Time)")
plt.xlabel("Month")
plt.ylabel("Total Sales (R$)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Step 2:Top 10 Product Categories by Sales
top_categories = (
    order_full.groupby('product_category_name')['total_price']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)
plt.figure(figsize=(10, 6))
sns.barplot(x=top_categories.values, y=top_categories.index)
plt.title("Top 10 Product Categories by Sales")
plt.xlabel("Total Revenue (R$)")
plt.ylabel("Product Category")
plt.tight_layout()
plt.show()

# Step 3: Orders by State
orders_by_state = (
    order_full.groupby('customer_state')['order_id']
```

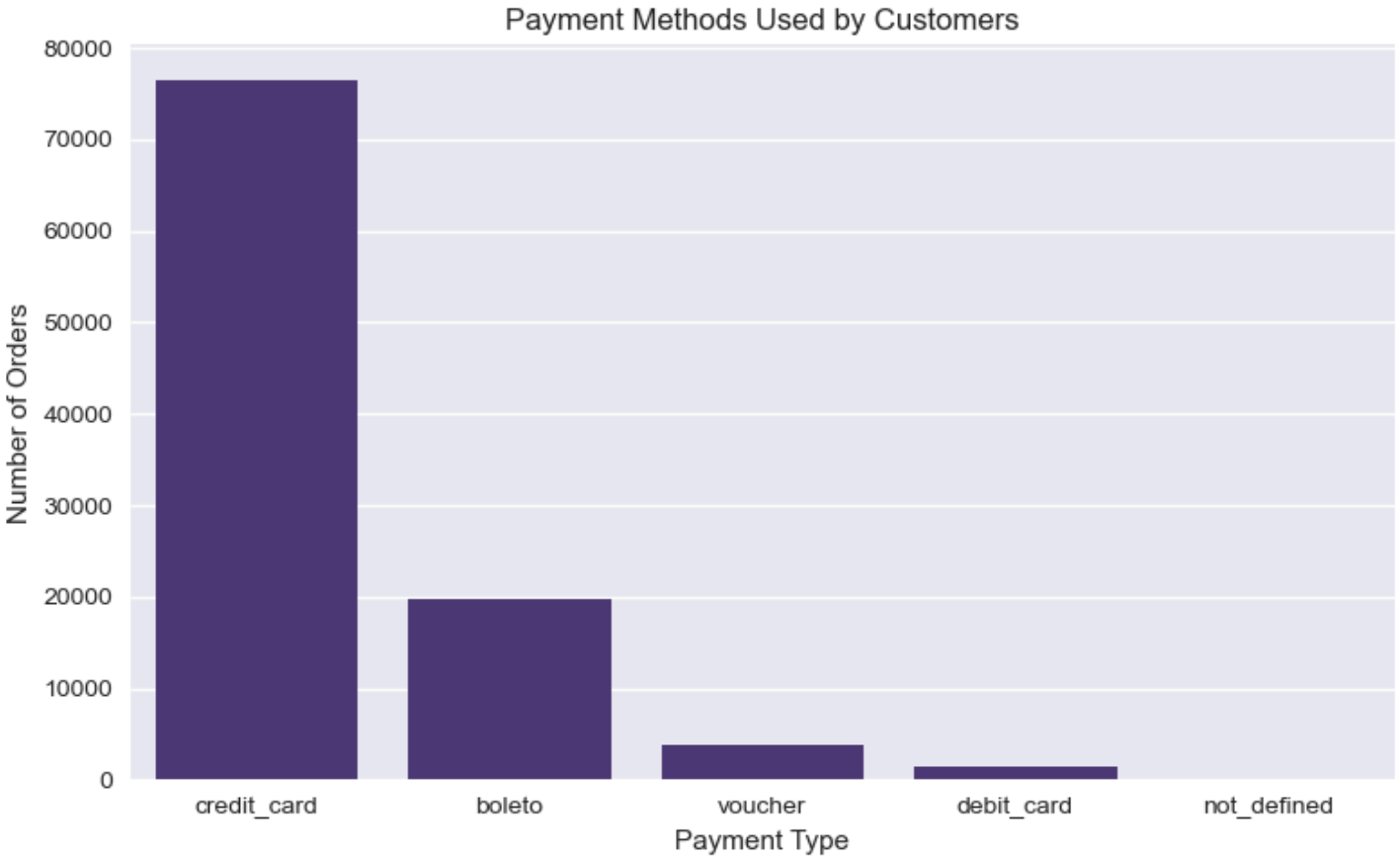
```
    order_full.groupby('customer_state')['order_id']
    .nunique()
    .sort_values(ascending=False)
    .head(10)
)
plt.figure(figsize=(10, 6))
sns.barplot(x=orders_by_state.values, y=orders_by_state.index)
plt.title("Top 10 States by Number of Orders")
plt.xlabel("Number of Orders")
plt.ylabel("State")
plt.tight_layout()
plt.show()

# Step 4: Delivery Days Distribution
plt.figure(figsize=(8, 5))
sns.histplot(order_full['delivery_days'].dropna(), bins=30, kde=True)
plt.title("Distribution of Delivery Duration (Days)")
plt.xlabel("Delivery Days")
plt.ylabel("Number of Orders")
plt.tight_layout()
plt.show()

# Step 5: Payment Type Distribution
payment_type = (
    order_full.groupby('payment_type')['order_id']
    .nunique()
    .sort_values(ascending=False)
)
plt.figure(figsize=(8, 5))
sns.barplot(x=payment_type.index, y=payment_type.values)
plt.title("Payment Methods Used by Customers")
plt.xlabel("Payment Type")
plt.ylabel("Number of Orders")
plt.tight_layout()
plt.show()
```




Top 10 Product Categories by Sales
beleza_saude
relogios_presentes
cama_mesa_banho
esporte_lazer
informatica_acessorios
moveis_decoracao
utilidades_domesticas
cool_stuff
automotivo
ferramentas_jardim
Product Category
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1e6
Total Revenue (R$)

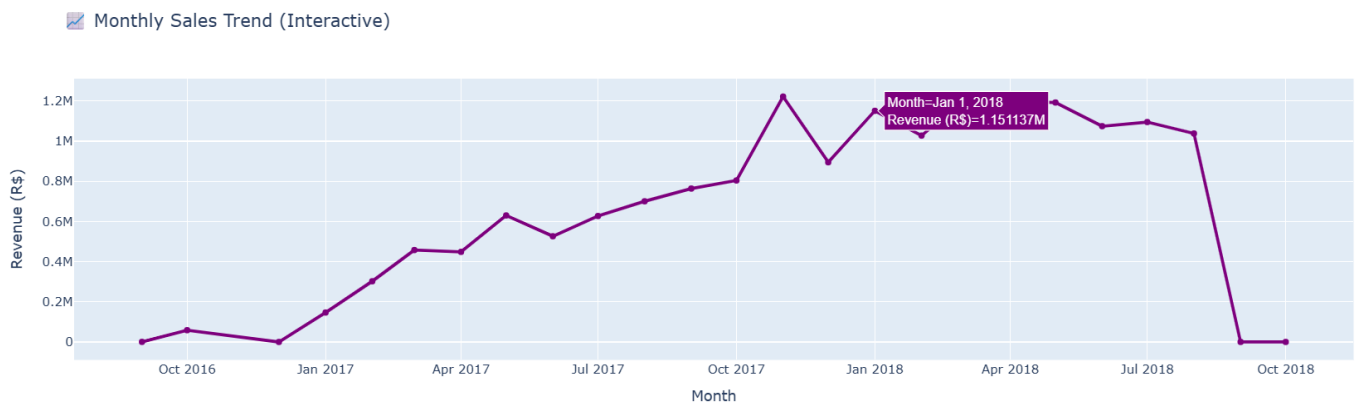

Top 10 States by Number of Orders
SP
RJ
MG
RS
PR
SC
BA
DF
ES
GO
State
0
5000
10000
15000
20000
25000
30000
35000
40000
Number of Orders

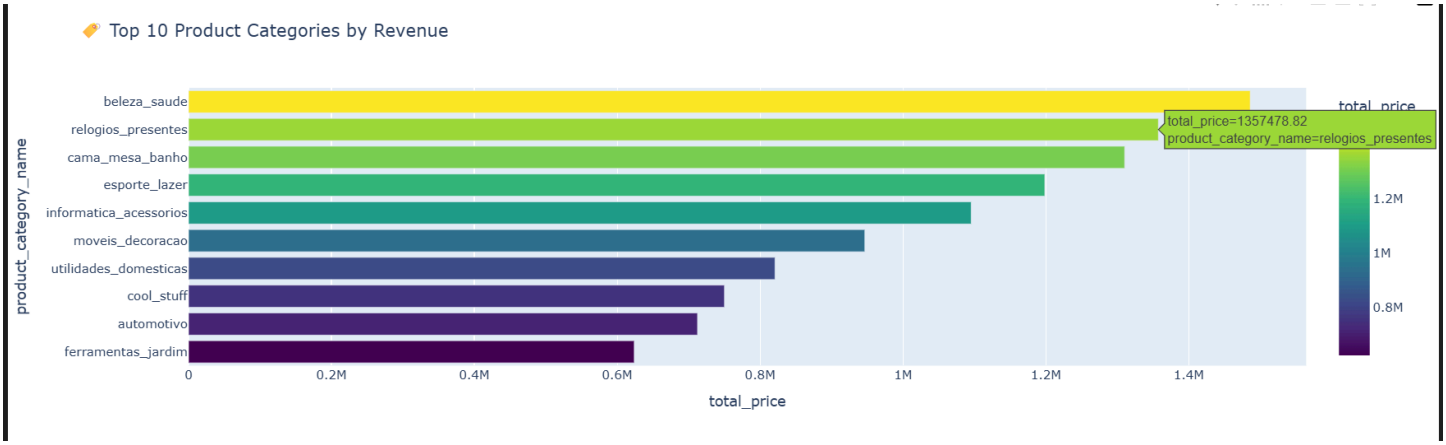

Distribution of Delivery Duration (Days)
Number of Orders
60000
50000
40000
30000
20000
10000
0
0
50
100
150
200
Delivery Days

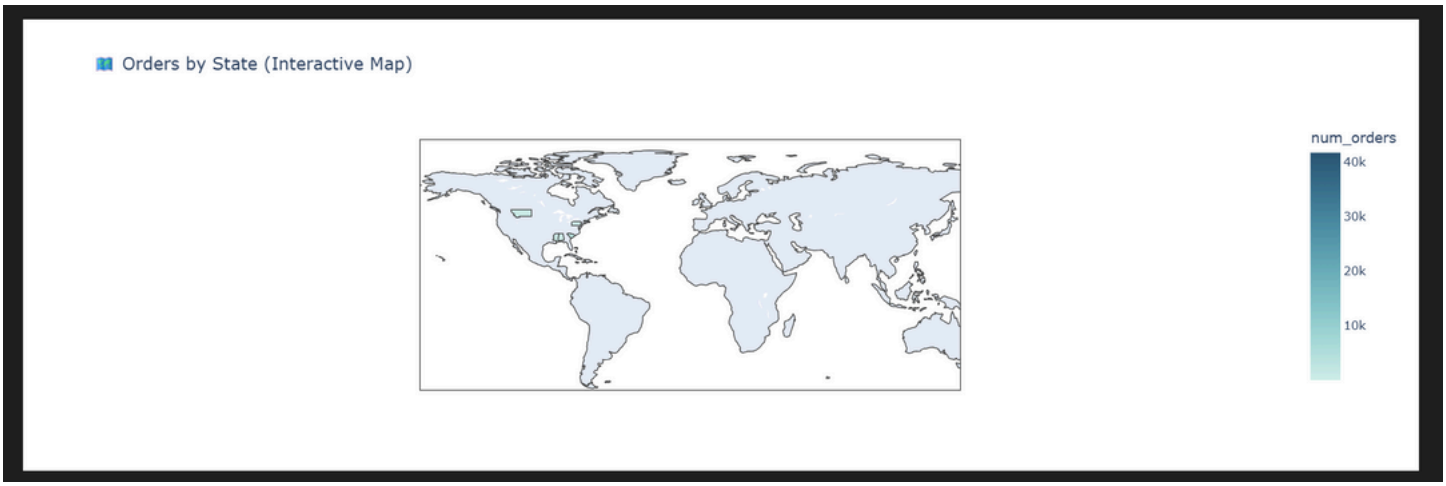

Payment Methods Used by Customers
Number of Orders
80000
70000
60000
50000
40000
30000
20000
10000
0
credit_card
boleto
voucher
debit_card
not_defined
Payment Type

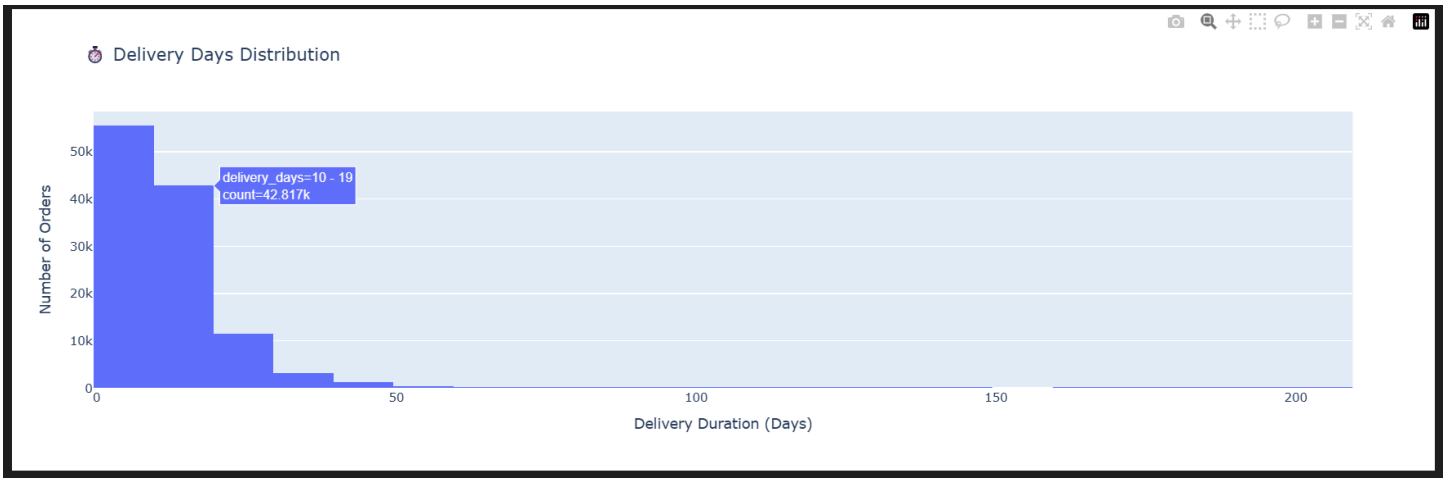
```
fig2 = px.bar(
    top_categories,
    x='total_price',
    y='product_category_name',
    orientation='h',
    title='🏈 Top 10 Product Categories by Revenue',
    color='total_price',
    color_continuous_scale='viridis'
)
fig2.update_layout(yaxis=dict(autorange="reversed"))
fig2.show()


# Step 3: Orders by State (Map Visualization)
orders_by_state = (
    order_full.groupby('customer_state')['order_id']
    .nunique()
    .reset_index()
    .rename(columns={'order_id': 'num_orders'})
)

fig3 = px.choropleth(
    orders_by_state,
    locations='customer_state',
    locationmode='USA-states',  # works for Brazil states if converted later
    color='num_orders',
    title='🗺️ Orders by State (Interactive Map)',
    color_continuous_scale='teal'
)
fig3.show()
```

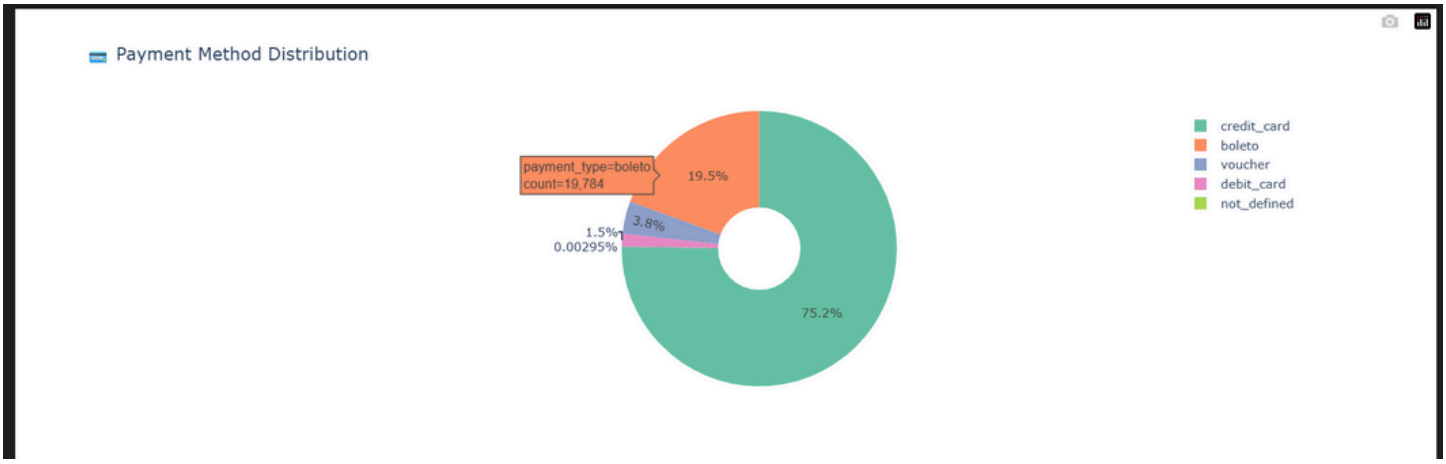
```
# Step 4: Delivery Days Distribution (Interactive Histogram)
fig4 = px.histogram(
    order_full,
    x='delivery_days',
    nbins=30,
    title='🔥 Delivery Days Distribution',
    color_discrete_sequence=['🟦'#636EFA']
)
fig4.update_xaxes(title='Delivery Duration (Days)')
fig4.update_yaxes(title='Number of Orders')
fig4.show()


# Step 5: Payment Type Share (Interactive Pie Chart)
payment_type = (
    order_full.groupby('payment_type')['order_id']
    .nunique()
    .reset_index()
    .rename(columns={'order_id': 'count'})
)

fig5 = px.pie(
    payment_type,
    values='count',
    names='payment_type',
    title='💳 Payment Method Distribution',
    color_discrete_sequence=px.colors.qualitative.Set2,
    hole=0.3
)
fig5.show()
```


Monthly Sales Trend (Interactive)
Month=Jan 1, 2018
Revenue (R$)=1.151137M
Revenue (R$)
1.2M
1M
0.8M
0.6M
0.4M
0.2M
0
Oct 2016
Jan 2017
Apr 2017
Jul 2017
Oct 2017
Jan 2018
Apr 2018
Jul 2018
Oct 2018
Month


Top 10 Product Categories by Revenue
product_category_name
beleza_saude
relogios_presentes
cama_mesa_banho
esporte_lazer
informatica_acessorios
moveis_decoracao
utilidades_domesticas
cool_stuff
automotivo
ferramentas_jardim
total_price=1357478.82
product_category_name=relogios_presentes
total_price
1.2M
1M
0.8M
0
0.2M
0.4M
0.6M
0.8M
1M
1.2M
1.4M
total_price


Orders by State (Interactive Map)
num_orders
40k
30k
20k
10k


Delivery Days Distribution
Number of Orders
50k
40k
30k
20k
10k
0
delivery_days=10 - 19
count=42.817k
0
50
100
150
200
Delivery Duration (Days)


Payment Method Distribution
payment_type=boleto
count=19,784
19.5%
3.8%
1.5%
0.00295%
75.2%
credit_card
boleto
voucher
debit_card
not_defined

(4)Business Insights & KPIs (with Interactive Charts)

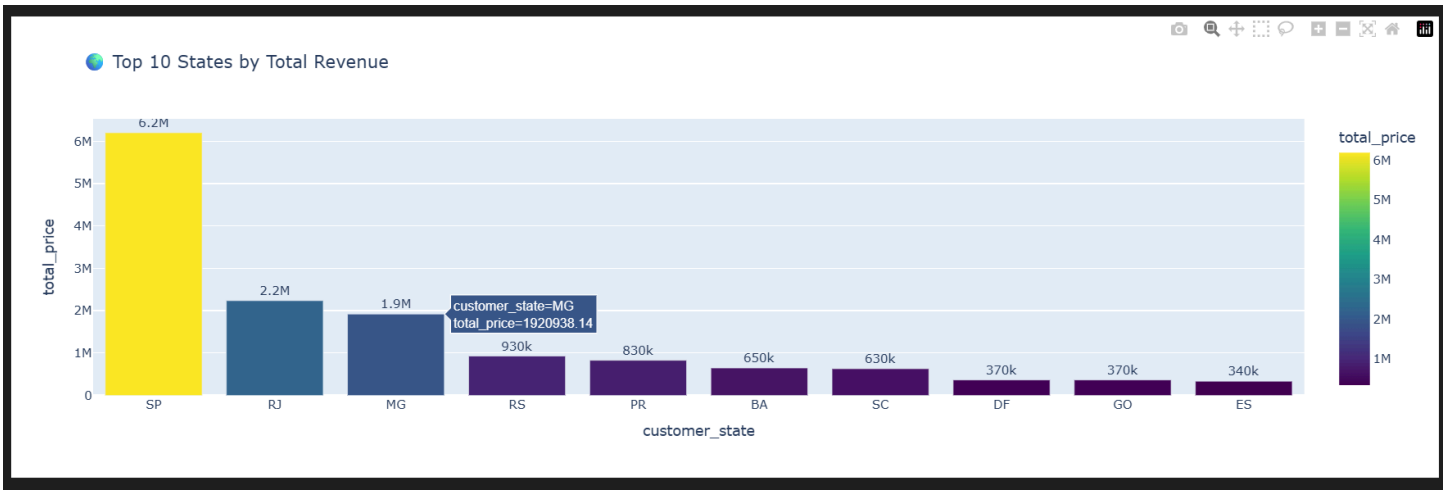
```
import plotly.express as px
import plotly.graph_objects as go

# step 1:Basic KPIs
total_revenue = order_full['total_price'].sum()
total_orders = order_full['order_id'].nunique()
avg_order_value = order_full.groupby('order_id')['total_price'].sum().mean()
unique_customers = order_full['customer_unique_id'].nunique()
unique_sellers = order_full['seller_id'].nunique()
avg_delivery_days = round(order_full['delivery_days'].mean(), 2)

# step 2: Display KPIs
kpi_fig = go.Figure()

kpi_fig.add_trace(go.Indicator(
    mode="number",
    value=total_revenue,
    title={"text": "Total Revenue (R$)"},
    number={"valueformat": ",.0f"}
))
kpi_fig.add_trace(go.Indicator(
    mode="number",
    value=avg_order_value,
    title={"text": "Avg Order Value (R$)"},
    domain={'x': [0.3, 0.7], 'y': [0, 1]}
))
kpi_fig.add_trace(go.Indicator(
    mode="number",
    value=total_orders,
```

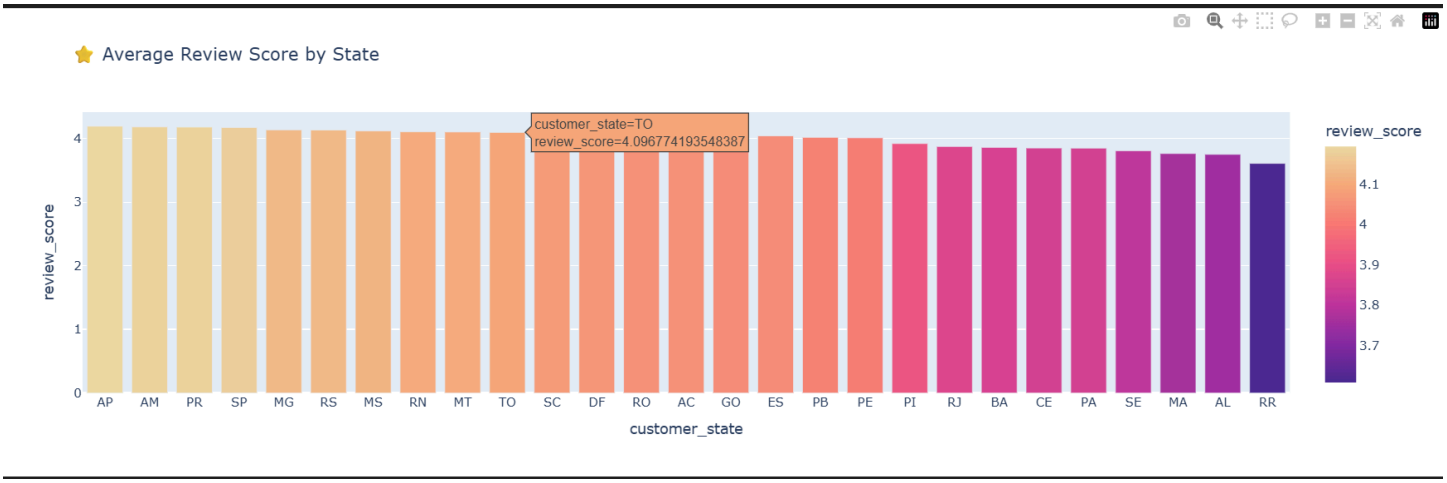
```
    mode="number",
    value=total_orders,
    title={"text": "Total Orders"},
    domain={'x': [0.7, 1], 'y': [0, 1]}
))

kpi_fig.update_layout(
    title="📊 Key Performance Indicators",
    grid={'rows': 1, 'columns': 3},
    template="plotly_dark"
)
kpi_fig.show()


# step 3: Top 10 States by Revenue
state_revenue = (
    order_full.groupby('customer_state')['total_price']
    .sum()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

fig_state = px.bar(
    state_revenue,
    x='customer_state',
    y='total_price',
    text='total_price',
    title='🌎 Top 10 States by Total Revenue',
    color='total_price',
    color_continuous_scale='Viridis'
)
fig_state.update_traces(texttemplate='%{text:.2s}', textposition='outside')
fig_state.show()
```

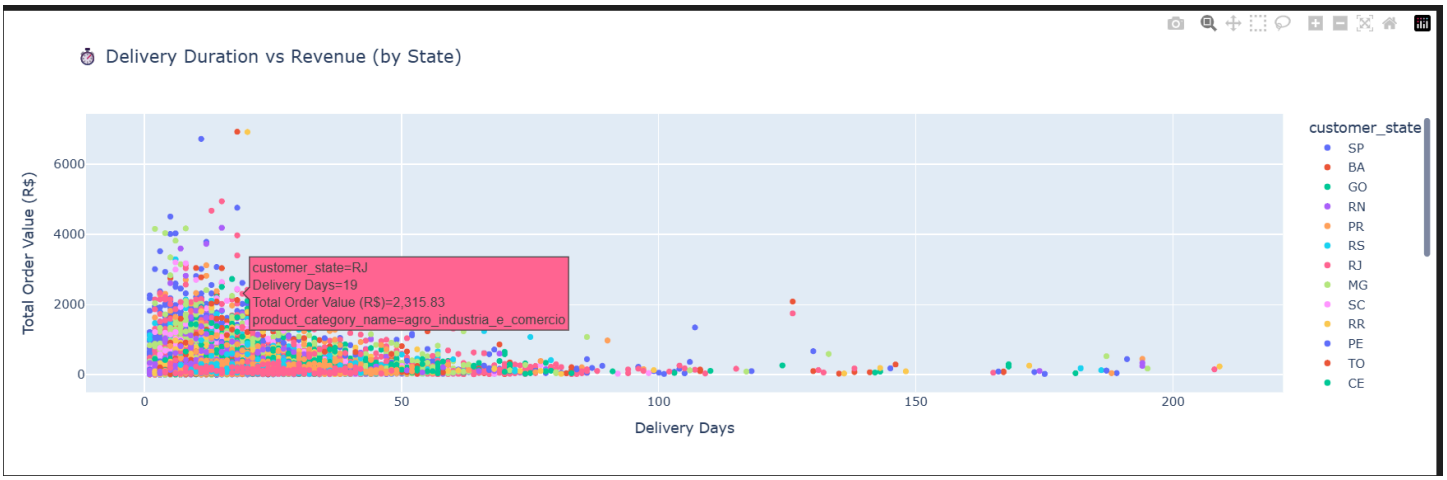
📊 Key Performance Indicators

Total Revenue (R$)	Avg Order Value (R$)	Total Orders
16,566,687	166.6	99.4k

```python
# step 4: Average Review Score by State
if 'review_score' in reviews.columns:
    order_reviews = pd.merge(orders, reviews, on='order_id', how='left')
    order_reviews = pd.merge(order_reviews, customers, on='customer_id', how='left')
    avg_review_state = (
        order_reviews.groupby('customer_state')['review_score']
        .mean()
        .reset_index()
        .sort_values(by='review_score', ascending=False)
    )

    fig_reviews = px.bar(
        avg_review_state,
        x='customer_state',
        y='review_score',
        title='⭐ Average Review Score by State',
        color='review_score',
        color_continuous_scale='Agsunset'
    )
    fig_reviews.show()


# step 5: Delivery Time vs. Total Revenue (Scatter)
fig_scatter = px.scatter(
    order_full,
    x='delivery_days',
    y='total_price',
    color='customer_state',
    title='🚚 Delivery Duration vs Revenue (by State)',
    labels={'delivery_days': 'Delivery Days', 'total_price': 'Total Order Value (R$)'},
    hover_data=['product_category_name']
)
fig_scatter.show()
```


Top 10 States by Total Revenue
total_price
6M
5M
4M
3M
2M
1M
0
6.2M
2.2M
1.9M
930k
830k
650k
630k
370k
370k
340k
customer_state=MG
total_price=1920938.14
SP
RJ
MG
RS
PR
BA
SC
DF
GO
ES
customer_state


Average Review Score by State
review_score
customer_state=TO
review_score=4.096774193548387
4.1
4
3.9
3.8
3.7
AP AM PR SP MG RS MS RN MT TO SC DF RO AC GO ES PB PE PI RJ BA CE PA SE MA AL RR
customer_state


Delivery Duration vs Revenue (by State)
Total Order Value (R$)
6000
4000
2000
0
customer_state=RJ
Delivery Days=19
Total Order Value (R$)=2,315.83
product_category_name=agro_industria_e_comercio
0
50
100
150
200
Delivery Days
customer_state
SP
BA
GO
RN
PR
RS
RJ
MG
SC
RR
PE
TO
CE

(5)Predictive Model – Late Delivery Prediction

```
# step 1: Import libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import plotly.express as px

# step 2: Prepare dataset
# Keep only rows with valid delivery dates
ml_data = order_full.dropna(subset=['order_delivered_customer_date', 'order_estimated_delivery_date'])

# Target variable: 1 = Late, 0 = On-time
ml_data['is_late'] = (ml_data['order_delivered_customer_date'] > ml_data['order_estimated_delivery_date']).astype(int)

# Select features
features = ['price', 'freight_value', 'delivery_days', 'payment_type', 'product_category_name']
ml_data = ml_data[features + ['is_late']].dropna()

# Encode categorical columns
label_enc = LabelEncoder()
for col in ['payment_type', 'product_category_name']:
    ml_data[col] = label_enc.fit_transform(ml_data[col].astype(str))

# step 3: Split dataset
X = ml_data.drop('is_late', axis=1)
y = ml_data['is_late']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# step 4: Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# step 5:Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

# step 6: Predictions
y_pred = model.predict(X_test_scaled)

# step 7: Evaluate
print("✅ Model Accuracy:", round(accuracy_score(y_test, y_pred)*100, 2), "%")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# step 8: Confusion Matrix Visualization
cm = confusion_matrix(y_test, y_pred)
cm_fig = px.imshow(
    cm,
    text_auto=True,
    color_continuous_scale='teal',
    title='Confusion Matrix: Late vs On-time',
    labels=dict(x="Predicted", y="Actual", color="Count")
)
cm_fig.show()

# step 9: Feature Importance
importances = pd.DataFrame({
    'Feature': X.columns,
    'Importance': model.feature_importances_
}).sort_values(by='Importance', ascending=False)
```

```python
fig_imp = px.bar(importances, x='Importance', y='Feature', orientation='h',
                 title='📊 Feature Importance - Late Delivery Prediction',
                 color='Importance', color_continuous_scale='viridis')
fig_imp.update_layout(yaxis=dict(autorange="reversed"))
fig_imp.show()
```

```
C:\Users\sarda\AppData\Local\Temp\ipykernel_5776\1378245365.py:17: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

✅ Model Accuracy: 95.86 %

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.99      0.98     20787
           1       0.81      0.65      0.72      1891

    accuracy                           0.96     22678
   macro avg       0.89      0.82      0.85     22678
weighted avg       0.96      0.96      0.96     22678
```